

INTERN TECH TALK

Perry Huang

@perry_huang

Summer 2013

DISQUS



SUMMER OVERVIEW

- **First week:** Got onboard
- **Weeks 2-3:** Project strace-explain
- **Week 4:** Learned about internals of Disqus, Django, Memcached, Cassandra, Redis, and more
- **Week 5-6:** Analyzed Memcached Usage at Disqus
- **Weeks 7-12:** Worked on Disqus and other things



FIRST WEEK: GOT ONBOARD

Onboarding was *difficult*, but I survived. As the only intern, I knew I had **huge** responsibilities.

1. I hardly knew anything about Disqus, but got a lot of help from the team.
"What's Promoted Discovery, Storm, Sentry, Jones, Gargoyle, and etc.?"
2. I wanted to fix Disqus. I was learning throughout the summer and made more of an impact each week.



WEEKS 2-3: STRACE - EXPLAIN

Worked on `strace-explain`, a Ruby gem that can be used to trace and analyze system calls for any user-level process. It reveals time being spent waiting for networked I/O from different resources. This can be used in debugging the Django dev server.

Source:

```
http://github.com/disqus/strace-explain/
```

Install:

```
$ gem install strace-explain
```

```
Disquuss-MacBook-Pro:~ $ strace-explain -h
```

```
Usage: strace-explain -p [PID]  
      or: strace-explain [command]
```

Options:

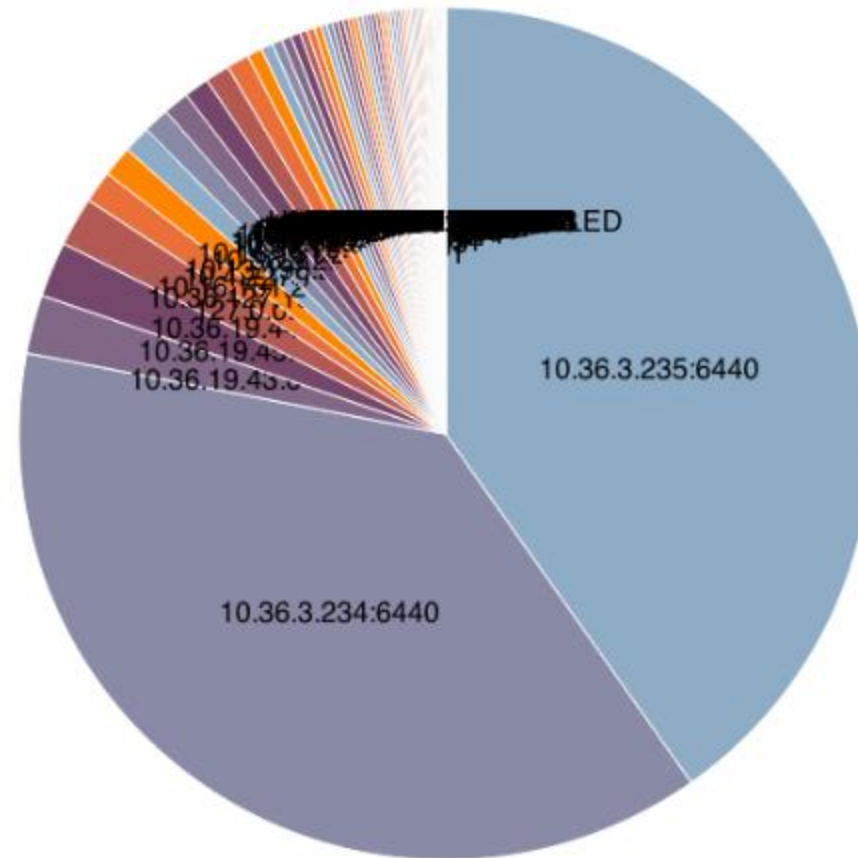
```
-p, --pid N  
-t, --time N  
-h, --help  
-v, --version
```

```
Attach to process PID N.  
Run analysis for N seconds.  
Show this message  
Show version
```



WEEKS 2-3: STRACE - EXPLAIN

<http://strace-explain.herokuapp.com/analysis/53558744580160>



CPU Time Spent for Each Service

#	Host:Port	Total Duration (secs.)
1	10.36.3.235:6440	2.7410590
2	10.36.3.234:6440	2.5754620

WEEK 4 AND BEYOND: LEARNING

Learned about the internals of various tools that we use at Disqus, as well as Disqus itself:

- Memcached and alternatives (e.g. mattproxy, mattdb, twitter-memcached, fatcache)
- Redis, Cassandra, PostgreSQL
- Indoor golf
- Disqus, Django, and of course, Python



DISQUS OPS TEAM

**{ Harder
Better
Faster
Stronger**

abhiytdara.deviantart.com



MEET THE DISQUS OPS TEAM

- Harder - Matt
- Better - John
- Faster - seems to be me
- Stronger - Mike



WEEK 5-6: ANALYZED MEMCACHED USAGE AT DISQUS

Created a tool to analyze Memcached traffic from real-time or previously recorded TCP/UDP dumps.

Source:

```
github link to memcached analysis tools
```

Results:

```
https://gist.github.com/perryh/07ef6828a9351f2604eb
```

At Disqus, most of our cache keys start with a group name, so we can adjust our code to optimize cache usage. Here are results from one group:

```
Type: :8:NewPostTreePaginator  
145810 gets = 9767 hits + 136043 misses  
153451 sets  
(1.2299291936654724% of total gets)  
(93.30155682051986% cache miss rate)  
(5.8092943561057195% of total sets)  
Min. payload length: 29
```



LESSONS LEARNED REGARDING CACHING

"Don't be retarded." ~ Ben

LESSON 1

Use the cache properly. Typically, we would first check our cache for our values before we query the database. If we do end up query the database, we would need to set the values into cache for them to be retrieved later. I found a few uses of the cache this summer where we did not actually set the keys we were querying.



LESSONS LEARNED REGARDING CACHING

LESSON 2

Avoid caching keys that have high cache miss rates or are never called within their lifetime. Large keys with this property are especially bad.

Case study from our Paginator:

```
Type: :8:NewPostTreePaginator
145810 gets = 9767 hits + 136043 misses
153451 sets
(1.2299291936654724% of total gets)
(93.30155682051986% cache miss rate)
(5.8092943561057195% of total sets)
Min. payload length: 29
  25th percentile: 29
  50th percentile: 2095
  75th percentile: 8518
Max. payload length: 273910
```

These keys can cause Memcached to evict other keys that are called more often within their lifetime. They are a waste of memory.



LESSONS LEARNED REGARDING CACHING

LESSON 3

Use `cache.get_many` and `cache.set_many` instead of looping `cache.get` and `cache.set`. This will dramatically decrease cache retrieval and setting from anywhere between 200 to 600 ms to a mere 60 ms on average.

Avoid this:

```
for o in objects:
    result[o.pk] = cache.get(make_key(o.pk))
# Build list of cache miss keys, query them from database
...
for missing in missing_objects:
    cache.set(make_key(missing.pk))
```

Do this:

```
cache_keys = [make_key(o.pk) for o in objects]
results = cache.get_many(cache_keys)
# Build list of cache miss keys, query them from database,
# and build a dict of cache miss keys and values
...
cache.set_many(missed_key_values)
```



WEEKS 7-END: SPEEDING UP DISQUS

CACHING OUR API RESPONSES

Using Varnish and the HTTP cache-control header helped decrease response time. While browsing New Relic, I noticed that some of our API endpoints were responding very slowly. New Relic reported that our 'Community' tab on large sites, such as CNN, would take upwards of 9 seconds to respond. This was confirmed by my own experience after experimenting on CNN, IGN, and The Atlantic.

With help from Matt, I got these API responses to respond in ~100 ms through the use of Varnish. This leads to a great improvement in user experience.



WEEKS 7-END: SPEEDING UP DISQUS

REMOVING KEYS THAT FILL UP CACHE

We used to have a key group that was cached in our cluster of Memcached servers. This proved to be really bad to cache:

```
Type: :8:html
1673394 gets = 624777 hits + 1048617 misses
1292953 sets
(14.11532907965599% of total gets)
(62.66408269660343% cache miss rate)
(48.94816303321555% of total sets)
Min. payload length: 32
  25th percentile: 70
  50th percentile: 122
  75th percentile: 255
Max. payload length: 29261
```

While a 62% miss rate is not the worst in the world, this key group did account for 48% of our cache sets. Removing this from our main Memcached cluster, along with some of my other fixes, allowed us to increase our cache hit ratio from 80% to over 90%.

We now have this key group in its own cache server being



WEEKS 7-END: THINGS THAT DID NOT WORK

REWRITE OUR PAGINATOR IN C++

I spent a lot of time learning to use SciPy Weave and Python Boost, but found the idea of rewriting the Paginator in C++ impractical. We have other plans in the pipeline to improve the Paginator.



TL;DR

- learned a lot this summer
- made performance upgrades to Disqus
- made open source contributions



THANKS

- everyone in operations team for helping me throughout the summer
- everyone for being awesome
- Mike for being my mentor and organizing an memorable sailing trip
- Jason and Daniel for forever branding me with my Disqus tattoo

